AIT
Hi-tech team

Введение в систему контроля версий

Занятие 5

VERSION CONTROL SYSTEM





Что такое система контроля версий?

Система контроля версий (VCS) — это инструмент, который **отслеживает каждое изменение в коде** с течением времени. Она фиксирует, *кто*, *когда* и *что* изменил в проекте.

📁 Хранение истории

Полная летопись всех изменений в проекте

👥 Командная работа

Несколько разработчиков работают над одним кодом

🔄 Откат изменений

Возврат к любой предыдущей версии в любой момент

Зачем нужен Git?



Git — ваша страховая сеть

Git решает три ключевые проблемы каждого разработчика:

- **Сохранение истории**
  Каждый коммит — это снимок проекта в конкретный момент времени
- **Защита от потери данных**
  Случайно удалили файл? Git восстановит любую версию
- **Откат к любой версии**
  Новая функция сломала проект — откатитесь за секунды

Локальный и удалённый репозиторий

💻 Локальный репозиторий

Находится прямо на вашем компьютере. Вы работаете с кодом, создаёте коммиты и управляете историей проекта **без подключения к интернету**.

ℹ Все изменения сначала фиксируются локально

☁️ Удалённый репозиторий

Хранится на сервере — например, на **GitHub**. Обеспечивает резервное копирование, совместный доступ и синхронизацию между разработчиками.

✓ Ваш код в безопасности даже при поломке ПК



AIT
Hi-tech team

Модель работы Git

Прежде чем использовать команды, важно понять три ключевые зоны Git:

1

Working Directory

Рабочая папка, где вы редактируете файлы проекта

2

Staging Area

«Зал ожидания» — файлы, подготовленные для коммита

3

Commit

Сохранённый снимок проекта, добавленный в историю

Staging Area — уникальная особенность Git, позволяющая точно выбирать, какие изменения войдут в следующий коммит.

Базовые команды

`git init`

Инициализирует новый репозиторий в текущей папке. Создаёт скрытую директорию `.git`

`git status`

Показывает состояние файлов: какие изменены, какие добавлены в staging, какие не отслеживаются

`git add`

Добавляет файлы в staging area. Используйте `git add .` для всех изменений

`git commit -m ""`

Фиксирует изменения из staging area с описательным сообщением



Работа с удалённым репозиторием



Синхронизация с GitHub

Две команды связывают локальную работу с удалённым репозиторием:

`git push`

Отправляет ваши коммиты на удалённый сервер. Делитесь кодом с командой

`git clone`

Создаёт локальную копию удалённого репозитория со всей историей изменений

После клонирования проекта он уже связан с удалённым репозиторием автоматически

Практический сценарий

Базовый цикл работы с Git на реальном примере:

01

Изменить файл

Отредактируйте `index.html` или любой другой файл проекта в рабочей директории

02

Добавить в Staging

Выполните `git add index.html`, чтобы подготовить изменения к сохранению

03

Создать коммит

Зафиксируйте снимок командой `git commit -m "Обновил шапку сайта"`

04

Отправить на GitHub

Опубликуйте изменения командой `git push origin main`



Итоги занятия

Git = Управление версиями

Полная история изменений,

откат в любой момент,

защита от ошибок

Удалённый репозиторий = Безопасность

GitHub хранит ваш код в

облаке и открывает

командный доступ

Следующий шаг

Практикуйте ежедневно: `add` → `commit` → `push`